# Lab 09

6LoWPAN: Headers, Compression and Fragmentation

Communication Technologies for the Internet of Things | Lecture 09

## Aim and context

Lecture 09 presents 6LoWPAN as the adaptation layer that makes it possible to carry IPv6 in 127-octet IEEE 802.15.4 frames. This lab reproduces, with a small Python program, the calculations and mechanisms of the lecture: the budget of a frame, header compression, fragmentation and the construction of the interface identifier.

The work needs no radio, no hardware and no virtual machine: it runs on any computer with Python 3. It is meant to be done during the class, at each student's pace, and is **not an assignment to hand in**.

> **Note**
>
> The values are computed, not measured on a real radio. The compressed bytes are, however, interpreted by the `scapy` library as 6LoWPAN headers, which lets their validity be checked.

## Learning outcomes

By the end of this lab, the student should be able to:

- Compute how many payload octets are left in an 802.15.4 frame, with and without security.
- Identify the fields of the IPHC header and of the UDP NHC and relate them to the maximum compression of 6 octets.
- Compare the cost of compression with link-local addresses and with global addresses.
- Estimate the number of frames needed for a message and the effect of loss on fragmentation.

- Build the interface identifier from the PAN identifier and the short address.

## Requirements

- A computer running Windows, macOS or Linux, with Python 3.9 or later.
- An Internet connection to install `scapy`.
- Estimated duration: 40 to 60 minutes.

## How to use this guide

Each step presents, in this order, the aim, the context, the theoretical background, the commands to run and the expected result. The boxes always have the same meaning:

**Command to run**

```
Command to run in the terminal.
```

**Expected result (values may vary)**

```
Result that should be displayed.
```

**Analyse**

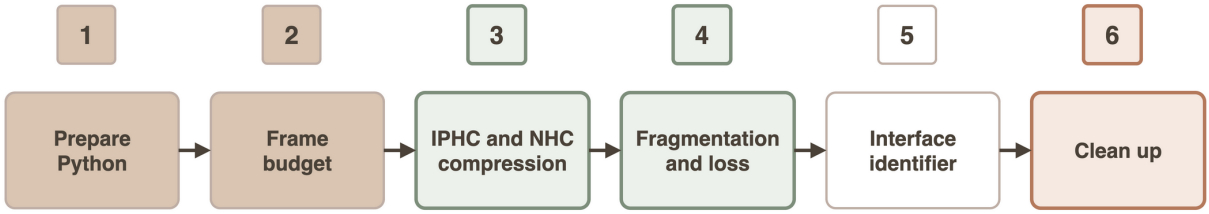
A moment to pause and interpret the result, on your own or in discussion with colleagues.

**Possible error and fix**

An error that may occur and how to fix it.

The results shown were obtained with the program in this guide and should match the student's.

## Overview




Everything runs in a single Python program; the numbers are the ones on the slides

**How to read the figure**

The six steps read from left to right. The first two, in beige, prepare the program and compute the frame budget. Steps 3 and 4, in green, are the core: header compression and fragmentation. Step 5, in white, builds the interface identifier, and step 6 closes the work.

## Step 1 — Preparing Python

**Aim.** Create an isolated environment, install `scapy` and save the lab program.

**Theoretical background**

A Python virtual environment is a folder that holds an isolated copy of the interpreter and the libraries. `scapy` is a packet manipulation library that knows the format of 6LoWPAN headers, so it serves both to build IPv6 packets and to validate the bytes the program produces.

### 1.1 Environment and library

**Command to run**

```
python3 -m venv ~/tciot-lab09
~/tciot-lab09/bin/pip install scapy
```

On Windows, use:

**Command to run**

```
python -m venv tciot-lab09
tciot-lab09\Scripts\pip install scapy
```

### 1.2 The program

Create the file `sixlowpan_lab.py` with the contents below. The file is also available in the `code` folder of the lecture material.

**Contents of file `sixlowpan_lab.py`**

```
# /// script
# requires-python = ">=3.9"
# dependencies = ["scapy"]
# ///
"""6LoWPAN lab (Lecture 09): header budget, IPHC/NHC compression, fragmentation,
    interface identifier.

Run:  uv run sixlowpan_lab.py [payload_bytes]
```

```
"""
import logging
import sys
from math import ceil

logging.getLogger("scapy.runtime").setLevel(logging.CRITICAL)
from scapy.layers.inet6 import IPv6, UDP
from scapy.layers.sixlowpan import LoWPAN_IPHC

FRAME = 127                 # IEEE 802.15.4 maximum frame size
MAC_OPEN, MAC_SEC = 25, 46  # MAC header without / with AES-CCM-128


def budget(ipv6_udp_header, mac):
    """Octets left for the application payload in one frame."""
    return FRAME - mac - ipv6_udp_header


def compress(link_local=True, ports=(0xF0B1, 0xF0B2)):
    """Minimal IPHC + NHC(UDP) encoder, returns the compressed header bytes.

    link_local=True : both addresses elided (derived from the link-layer addresses
    )
    link_local=False: both addresses carried inline (128 bits each)
    """
    hlim = 0b10                                     # hop limit 64 elided
    b0 = (0b011 << 5) | (0b11 << 3) | (1 << 2) | hlim   # 011 | TF=11 elided | NH
    =1 NHC follows | HLIM
    sam = dam = 0b11 if link_local else 0b00
    b1 = (0 << 7) | (0 << 6) | (sam << 4) | (0 << 3) | (0 << 2) | dam
    out = bytes([b0, b1])
    if not link_local:
        out += bytes(32)                            # placeholder for the two inline
     addresses
    nhc = 0b11110_0_11                              # UDP NHC, checksum inline, both
     ports 4 bits
    out += bytes([nhc, ((ports[0] & 0xF) << 4) | (ports[1] & 0xF)]) + b"\x00\x00"
    return out


def fragments(compressed_header, payload, mac):
    """Number of 802.15.4 frames needed (simplified: FRAG1 = 4 octets, FRAGN = 5
    octets of overhead)."""
    data = len(compressed_header) + payload
    if data <= FRAME - mac:
        return 1
    first = ((FRAME - mac - 4) // 8) * 8
    rest = ((FRAME - mac - 5) // 8) * 8
```

```
    return 1 + ceil((data - first) / rest)


def short_to_iid(pan, short):
    """PAN:0000:short -> interface identifier (FFFE in the middle, U/L bit set to
    0 = local)."""
    b = pan.to_bytes(2, "big") + b"\x00\x00" + short.to_bytes(2, "big")
    iid = bytearray(b[:3] + b"\xff\xfe" + b[3:])
    iid[0] &= 0xFD
    return ":".join(f"{iid[i]:02x}{iid[i+1]:02x}" for i in range(0, 8, 2))

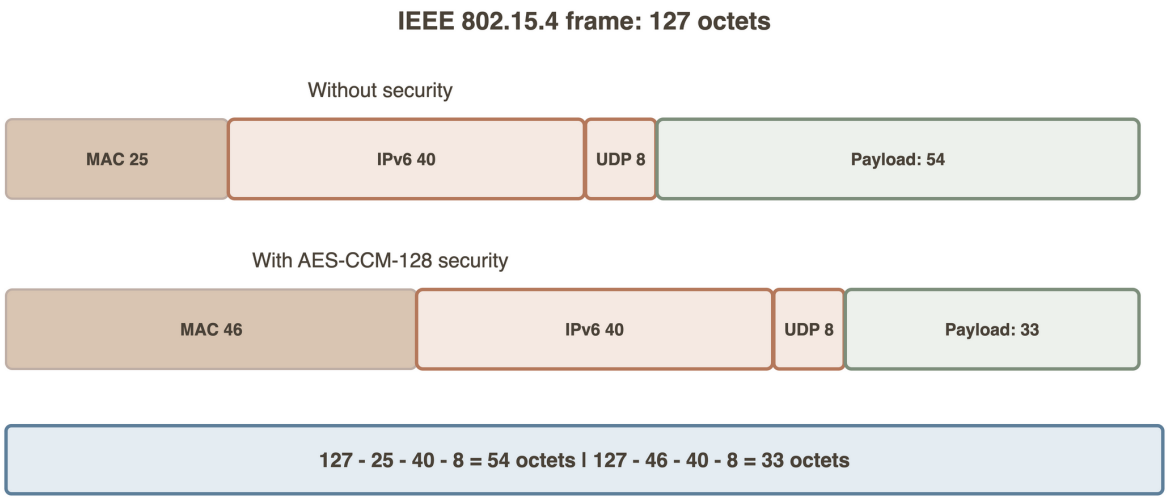

if __name__ == "__main__":
    app = int(sys.argv[1]) if len(sys.argv) > 1 else 20
    pkt = IPv6(src="fe80::1", dst="fe80::2") / UDP(sport=5683, dport=5683) / (b"x"
     * app)
    hdr = len(pkt) - app
    print(f"IPv6 + UDP header: {hdr} octets; application payload: {app} octets")
    for name, mac in (("without security", MAC_OPEN), ("with AES-CCM-128", MAC_SEC
    )):
        print(f"  frame budget {name}: {budget(hdr, mac)} octets left")
    for ll in (True, False):
        c = compress(link_local=ll)
        kind = "link-local" if ll else "global addresses inline"
        print(f"IPHC+NHC ({kind}): {len(c)} octets  ->  {budget(len(c), MAC_OPEN)}
     octets left")
    c = compress(True)
    print("bytes:", c.hex(" "))
    print("scapy sees IPHC:", LoWPAN_IPHC(c).summary())
    for size in (20, 60, 150, 400):
        n_u = fragments(bytes(hdr), size, MAC_OPEN)
        n_c = fragments(compress(True), size, MAC_OPEN)
        print(f"payload {size:3d} octets: {n_u} frame(s) uncompressed, {n_c}
    compressed")
    for p in (0.01, 0.1):
        print(f"loss {p:.0%} per frame: success with 1 frame {(1-p):.2f}, with 5
    frames {(1-p)**5:.2f}")
    print("interface id for PAN 0xABCD, short 0x0001:", short_to_iid(0xABCD, 0
    x0001))
```

**Possible error and fix**

**ModuleNotFoundError: No module named 'scapy'**: the program was run with the system Python and not the virtual environment's. Use `~/tciot-lab09/bin/python sixlowpan_lab.py`.

## Step 2 — The frame budget

**Aim.** Compute how many octets are left for the application in an 802.15.4 frame.




### How to read the figure

The figure is the same as on the slide. The bars have a width proportional to the number of octets: the MAC header, IPv6 and UDP in beige and red, and the payload in green. The upper bar has no security and the lower one uses AES-CCM-128, which widens the MAC header from 25 to 46 octets.

### Theoretical background

The 802.15.4 frame has at most 127 octets. The IPv6 header takes 40 octets and UDP takes 8, with no compression. Subtracting the MAC header from these values gives 54 payload octets without security and 33 with AES-CCM-128 security. The `budget` function in the program does exactly this sum.

**To analyse in this step:** the proportion between header and data. More than half of the frame is header.

### Command to run

```
~/tciot-lab09/bin/python sixlowpan_lab.py 20
```

**Expected result (values may vary)**

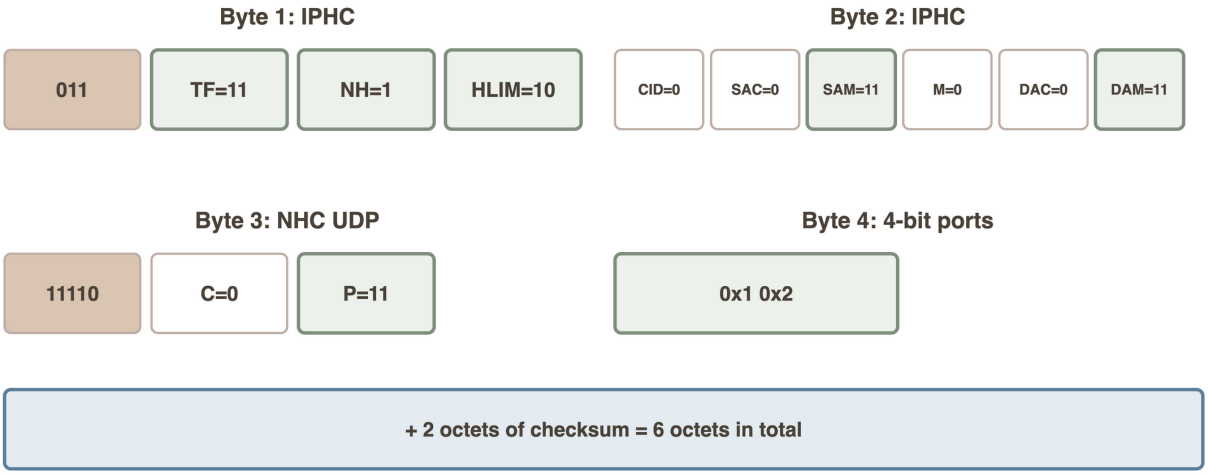
```
IPv6 + UDP header: 48 octets; application payload: 20 octets
  frame budget without security: 54 octets left
  frame budget with AES-CCM-128: 33 octets left
```

**Analyse**

Change the argument to 100 and repeat. Does the budget change? What would happen to the message with 100 payload octets?

## Step 3 — Compression with IPHC and NHC

**Aim.** Observe the bytes of the compressed header and compare the cost with link-local addresses and with global addresses.




### How to read the figure

The first two octets form the IPHC header. The first says that the traffic class and flow label field is omitted, that the next header uses NHC and that the hop limit 64 is omitted. The second says that both addresses are omitted, because they are derived from the layer-2 addresses. The third octet is the UDP NHC, and the fourth carries the 4-bit ports. The total, with the two checksum octets, is 6 octets.

### Theoretical background

Stateless compression omits everything the receiver can deduce: the version, the length, the usual hop limit and, on a link-local link, the addresses. UDP ports in the range 61616 to 61631 (`0xF0B0` to `0xF0BF`) fit in 4 bits each. When addresses are global, they cannot be deduced from layer 2 and must be carried, which raises the header size.

**To analyse in this step:** which fields disappeared and why the receiver can reconstruct them.

### Command to run

```
~/tciot-lab09/bin/python sixlowpan_lab.py 20 | sed -n 4,7p
```

**Expected result (values may vary)**

```
IPHC+NHC (link-local): 6 octets  ->  96 octets left
IPHC+NHC (global addresses inline): 38 octets  ->  64 octets left
bytes: 7e 33 f3 12 00 00
scapy sees IPHC: LoWPAN_IPHC / LoWPAN_NHC / IPv6 / UDP :::61617 > :::61618
```

**Analyse**

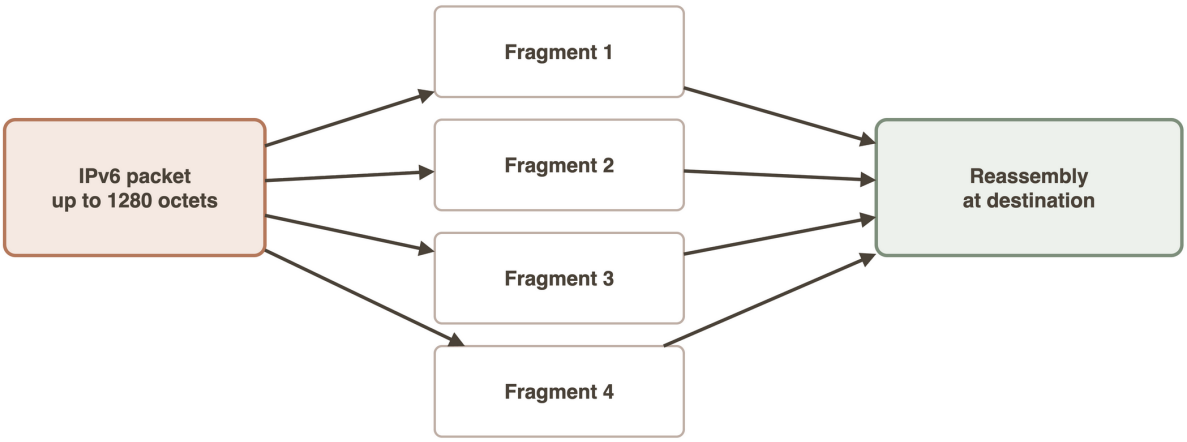
The `bytes` line shows the compressed header in hexadecimal. Match each octet to the diagram. Why do global addresses take the header from 6 to 38 octets?

**Possible error and fix**

On Windows, the `sed` command does not exist. Just run the program and read the corresponding lines.

## Step 4 — Fragmentation and loss

**Aim.** Estimate how many frames a message needs and the effect of frame loss.




Losing one fragment forces retransmission of the whole datagram

### How to read the figure

The IPv6 packet, on the left, is split into fragments, in the centre, which travel separately and are put back together at the destination, on the right. If a fragment is lost, the destination cannot rebuild the packet and the whole packet has to be retransmitted.

### Theoretical background

Each fragment adds 4 or 5 octets of fragmentation header, and the data space of each fragment is a multiple of 8 octets. The program uses a simplified calculation, which ignores details such as the offset of the first fragment. If the probability of losing a frame is $p$ and the message uses $n$ frames, the probability of delivery is $(1-p)^n$, which falls quickly with $n$.

**To analyse in this step:** the difference compression makes to the number of frames, and the effect that has on reliability.

### Command to run

```
~/tciot-lab09/bin/python sixlowpan_lab.py 20 | sed -n 8,13p
```

**Expected result (values may vary)**

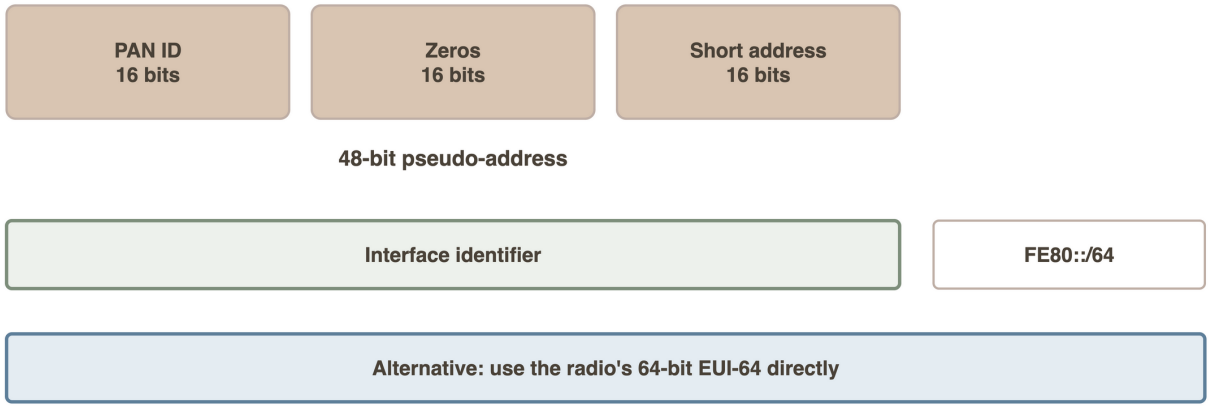
```
payload  20 octets: 1 frame(s) uncompressed, 1 compressed
payload  60 octets: 2 frame(s) uncompressed, 1 compressed
payload 150 octets: 3 frame(s) uncompressed, 2 compressed
payload 400 octets: 5 frame(s) uncompressed, 5 compressed
loss 1% per frame: success with 1 frame 0.99, with 5 frames 0.95
loss 10% per frame: success with 1 frame 0.90, with 5 frames 0.59
```

**Analyse**

For which message sizes does compression reduce the number of frames? With a 10% loss per frame, what reduction in reliability results from going from 1 to 5 frames?

## Step 5 — The interface identifier

**Aim.** Verify the construction of the interface identifier from the PAN identifier and the short address.




### How to read the figure

The figure shows the three blocks of the 48-bit pseudo-address: the PAN identifier, 16 zero bits and the short address. Below, the interface identifier and the link-local prefix `FE80::/64`.

### Theoretical background

When the device only has the short address, the interface identifier is built from the PAN, 16 zero bits and the short address, with `FFFE` inserted in the middle and the universal/local bit set to zero, meaning local scope. The result is not globally unique: it only has meaning inside the PAN.

**To analyse in this step:** where each part of the short address appears in the final identifier.

### Command to run

```
~/tciot-lab09/bin/python sixlowpan_lab.py | tail -1
```

### Expected result (values may vary)

```
interface id for PAN 0xABCD, short 0x0001: a9cd:00ff:fe00:0001
```

**Analyse**

At the end of the program, change the PAN identifier and the short address, and predict the result before running it. What link-local address do you get?

## Step 6 — Cleaning up

**Command to run**

```
rm -r ~/tciot-lab09
```

## Review questions

1. What fraction of a frame with security is payload when there is no compression?
2. Why does compression only reach 6 octets on a link-local link?
3. Why does stateless compression matter to devices with little memory?
4. With a 10% loss per frame, what is the reliability difference between a 1-frame message and a 5-frame message?
5. Why was IP not used in the lecture's case study?

## Summary

- The **budget** of an 802.15.4 frame is small: 54 payload octets without security and 33 with it.
- Stateless **compression** reduces 48 octets of header to 6 on a link-local link.
- With **global addresses** compression is partial, and the header grows.
- **Fragmentation** allows larger messages, but the probability of delivery falls with the number of frames.